

一、实验内容

本实验完成了 DevEco Studio 与 HarmonyOS 开发环境的配置，并以官方计算器示例为基础，使用 ArkTS 和 ArkUI 开发了一款多功能计算器。

在基本四则运算功能之上，本项目增加了科学计算、计算历史、存储器、单位换算、日期计算、主题色切换等功能，并针对按键布局、长数字显示、删除操作与动画反馈进行了个性化改进。应用最终在 DevEco Studio 的 X View 中运行成功，并通过 Hvigor 完整构建。

1. 开发环境与工程结构

实验在 Windows 64 位系统上完成，使用 DevEco Studio 创建 Empty Ability 工程。工程创建后完成 SDK 同步，并通过 X View 验证 ArkTS 页面能够正确编译和运行。

```
MyApplication
├─ AppScope/app.json5          # 应用包名与版本配置
├─ entry/src/main
│  ├─ ets/pages/HomePage.ets   # 界面与主要交互逻辑
│  ├─ ets/common/util          # 表达式计算工具
│  ├─ ets/viewmodel            # 基础按键数据模型
│  ├─ resources                # 图片、颜色和字符串资源
│  └─ module.json5             # EntryAbility 与设备配置
└─ build-profile.json5         # 构建配置
```

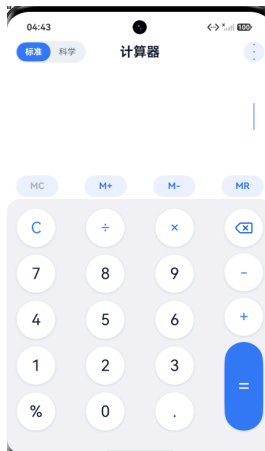
module.json5 将模块类型设置为 entry，并声明支持 Phone 设备：

```
{
  "module": {
    "name": "entry",
    "type": "entry",
    "mainElement": "EntryAbility",
    "deviceTypes": ["phone"],
    "deliveryWithInstall": true,
    "installationFree": false
  }
}
```

2. 标准计算器

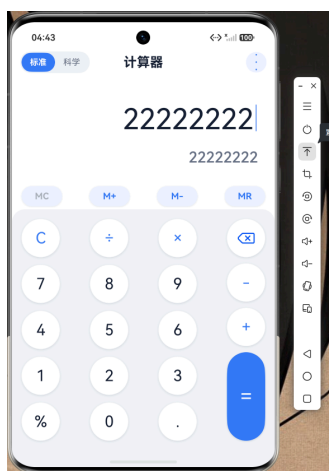
标准模式实现数字、小数、加、减、乘、除、百分号、清空、退格和等号计算。页面上方实时显示当前表达式和计算预览，按下等号后将结果写入主显示区域。

删除键支持短按逐位删除和长按清空。等号计算得到的结果被视为一个整体，再按删除键时可一次删除。数字过长时，显示字体逐级缩小；达到最小字号后，显示区域支持水平滚动并始终定位到光标末尾。

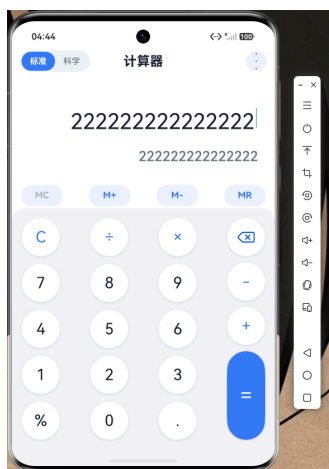


上图应用启动后的标准计算器界面。左上角可切换标准与科学模式，右上角三点按钮用于打开扩展功能。中部为输入及预览区域，下方依次为存储器按键和基础运算键盘。

输入数字时，末尾显示一条周期闪烁的竖线光标，预览结果同步显示在下一行。下图展示普通长度数字的输入状态：



当输入内容继续增加时，主显示与预览文字会自动递减字号，以尽可能在一行内显示完整内容；缩小到最低字号后，显示区允许左右滑动并自动跟随末尾光标。下图展示长数字触发字体缩小后的界面：



3. 科学计算器

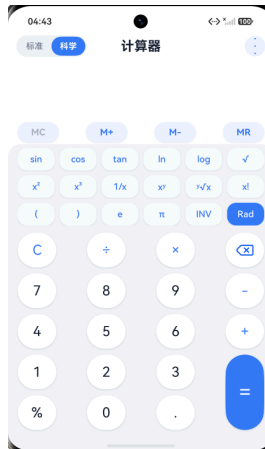
点击左上角“科学”可切换到科学模式。科学键盘采用 3 行×6 列布局，在单页中展示 18 个按键：

- 三角函数：sin、cos、tan；
- 对数与开方：ln、log、√；

- 幂与根： x^2 、 x^3 、 x^y 、 $\sqrt[y]{x}$ 、 $1/x$ ；
- 其他功能： $x!$ 、左右括号、常数 e 、常数 π 、**INV** 和 **Rad**。

INV 用于切换反函数。开启后，三个三角函数变为反正弦、反余弦和反正切，**ln** 与 **log** 分别变为 e^x 与 10^x 。
Rad 用于切换角度制和弧度制。表达式解析器支持括号优先级、幂运算和任意次根，并处理负数奇次根、零次根和阶乘范围等边界情况。

科学按键和基础按键经过统一排版。科学键高度、行间距和圆角被适当放大，基础按键区域向下铺开，减少底部空白，并保证 **sin**、**cos**、**tan** 始终可见。



上图为科学计算器界面。18 个科学功能键完整显示在同一页，当前处于弧度制，因此 **Rad** 按键以主题色高亮。

4. 表达式解析与精度处理

项目使用数组保存操作数、运算符和括号，通过运算符优先级将中缀表达式转换为后缀队列，再使用栈计算结果。加减乘除、幂和任意次根具有不同优先级，其中幂与根按右结合处理。

```
shouldPopOperator(current: string, stackTop: string): boolean {
    let currentPriority = this.getPriority(current);
    let stackPriority = this.getPriority(stackTop);
    let rightAssociative = current === CommonConstants.POWER ||
        current === CommonConstants.ROOT;
    return currentPriority < stackPriority ||
        (!rightAssociative && currentPriority === stackPriority);
}
```

任意次根按“先输入被开方数，再输入根指数”的方式计算。例如输入 $8 \sqrt[3]{}$ 得到 2。对于负数的奇次根，程序单独保留负号：

```
root(radicand: number, degree: number): number {
    if (degree === 0) return Number.NaN;
    if (radicand < 0 && degree === Math.floor(degree) &&
        Math.abs(degree % 2) === 1) {
        return -Math.pow(-radicand, 1 / degree);
    }
    return Math.pow(radicand, 1 / degree);
}
```

结果统一保留 12 位有效数字，并过滤 **NaN** 与正负无穷，避免浮点数尾部出现过多无意义数字。

5. 三点菜单与主题色切换

为减少顶部按钮拥挤，历史记录、单位换算、日期计算和主题设置统一收纳到右上角三点菜单。点击按钮时菜单带旋转和淡入过渡，点击遮罩可关闭菜单。用户从这里进入各项扩展功能，下面按照菜单顺序分别介绍历史记录、单位换算和日期计算。

主题提供蓝、紫、绿、橙四种颜色。主题索引通过 `PersistentStorage` 保存，重新打开应用后仍使用上次选择的主题。模式切换按钮、运算符、等号、科学功能键和各弹窗重点内容都会随主题统一变化。

下图为右上角三点扩展菜单。菜单集中提供历史记录、单位换算、日期计算和四种主题色选择；图中已切换为紫色主题，界面的模式按钮、功能文字和运算按键同时更新配色。



```
PersistentStorage.persistProp<number>('calculatorTheme', 0);
private themeColors: Array<string> = [
    '#3478F6', '#7C4DFF', '#00A86B', '#F26A2E'
];

primaryColor(): string {
    return this.themeColors[this.themeIndex];
}
```

6. 历史记录与存储器

每次按等号或执行单目科学运算后，程序将“表达式 = 结果”保存到历史记录。新记录显示在最上方，最多保留 30 条，历史面板支持一键删除全部记录。历史数据通过 `PersistentStorage` 持久化，应用重新运行后仍然存在。

```
PersistentStorage.persistProp<Array<string>>('calculatorHistory', []);

addHistory(expression: string, result: string): void {
    let newHistory: Array<string> = Array.from(this.historyRecords);
    newHistory.unshift(expression + ' = ' + result);
    if (newHistory.length > 30) newHistory.pop();
    this.historyRecords = newHistory;
}
```

计算器还提供大写的 `MC`、`M+`、`M-`、`MR` 四个存储器按键。`M+` 和 `M-` 对当前值进行累加或累减，`MR` 读取存储值，`MC` 清除存储值。存储器同样使用 `PersistentStorage` 保存。

历史记录以弹窗形式覆盖在计算器上方，每条记录保留原表达式和计算结果，底部提供醒目的“清空全部记录”按钮：



7. 单位换算

右上角三点菜单可打开单位换算面板。换算功能包含长度、质量、温度、面积、体积和存储六类单位。用户选择源单位和目标单位后输入数值，结果会实时显示，也可一键交换两个单位。

换算时先将输入值转换为该类别的基准单位，再转换为目标单位。温度因存在偏移量而使用单独公式，例如摄氏度转换为华氏度使用 $F = C \times 9 / 5 + 32$ 。

下图展示温度换算界面。顶部可切换六种换算类别，中部选择摄氏度和华氏度，底部使用独立数字键盘输入数值，并可执行交换、删除、正负号和完成操作。



8. 日期计算

日期计算面板使用 ArkUI 的 `DatePicker` 选择真实日期，提供“日期间隔”和“距离某日”两种模式。前者计算两个日期之间相差的天数；后者显示目标日期还有多少天、已经过去多少天，或提示目标日期就是今天。

程序将所选年月日转换成 UTC 日序号后再作差，避免时区或夏令时造成一天偏差：

```
calendarDayNumber(value: Date): number {
  return Math.floor(Date.UTC(
    value.getFullYear(), value.getMonth(), value.getDate()
  ) / 86400000);
}
```

该方法会按照现实日历自动处理不同月份的天数和闰年。

下图展示“日期间隔”模式。用户分别滚动选择开始日期和结束日期，结果区实时显示相差天数：



下图展示“距离今天”模式。选择目标日期后，程序会根据设备当前本地日期判断该日尚未到来、已经过去或就是今天，并显示相应天数：



9. 动画与界面优化

所有按键均增加淡阴影和按压缩放效果。普通输入过程不播放位移动画，只显示周期闪烁的竖线光标；按下等号后，结果从下向上弹出并轻微回落。

```
animateResult(): void {
  this.resultOpacity = 0.18;
  this.resultOffset = 30;
  this.uiContext.animateTo({ duration: 210, curve: Curve.EaseOut }, () => {
    this.resultOpacity = 1;
    this.resultOffset = -5;
  });
  setTimeout(() => {
    this.uiContext.animateTo({ duration: 110, curve: Curve.EaseIn }, () => {
      this.resultOffset = 0;
    });
  }, 210);
}
```

长数字显示字号根据字符数动态计算，达到最小字号后改为横向滚动：

```
inputFontSize(): number {
    let baseSize = this.isScientific ? 38 : 48;
    let minSize = this.isScientific ? 24 : 26;
    let shrinkStart = this.isScientific ? 8 : 9;
    let extraLength = Math.max(0, this.inputValue.length - shrinkStart);
    return Math.max(minSize, baseSize - extraLength * 2);
}
```

10. 运行与构建验收

项目在 DevEco Studio 中完成工程同步，并使用 X View 检查标准模式、科学模式和弹窗功能。最后执行 Hvigor 的 `assembleHap` 任务，ArkTS 编译、资源处理与 HAP 打包均成功：

```
BUILD SUCCESSFUL
Finished :entry:default@CompileArkTS
Finished :entry:default@PackageHap
Finished :entry:default@assembleHap
```

二、问题总结与体会

问题及解决

1. 模拟器无法获取 API Version

最初启动模拟器时出现错误码 00401018，提示“获取设备 `apiVersion` 失败”。在模拟器暂时无法联机的情况下，改用 DevEco Studio 的 X View 进行界面与交互验证，项目能够成功编译运行。这说明运行目标、SDK 版本与设备镜像之间需要保持兼容。

2. 结果显示区域上下位置不协调

初始版本中主输入和预览结果靠近按键区域，视觉上出现上下不对齐。解决方式是分别设置主输入区域与预览文字的高度、行高、内边距和垂直居中方式，并为标准模式与科学模式设置不同的显示区高度。

3. 等号结果动画方向不自然

早期动画产生了从左向右移动的观感。调整后只在按下等号时设置 Y 轴位移，使结果从下向上弹出，再轻微回落到最终位置；普通数字输入不再触发位移动画。

4. 科学按键拥挤或部分按键不可见

科学功能增加后，曾出现 `cos`、`tan` 不可见以及按键挤在一起的问题。最终将 18 个按键按功能重新排序为 3 行×6 列单页布局，并调整按键高度、间距、圆角和整个键盘区的垂直位置，使所有功能同时可见，底部空白也明显减少。

5. 任意次根符号与参数顺序容易混淆

任意次根在输入、显示和栈求值时的参数顺序不同，曾出现根指数和被开方数颠倒的问题。解决方式是明确规定先输入被开方数、后输入根指数，并在计算工具中按照栈弹出顺序交换参数。同时让按键显示 $\sqrt[n]{x}$ ，表达式区域只显示清晰的根运算符。

6. 计算结果的删除方式不符合预期

最初按等号得到的结果仍按普通输入逐位删除。程序增加 `lastBtnIsEqu` 状态区分“正在输入”和“刚得到结果”：普通输入短按退格逐位删除，等号结果短按退格一次清空，长按退格在任何情况下都清空当前内容。

7. 长数字超出屏幕

固定字号无法容纳较长表达式。解决方式是根据字符数逐步缩小字体并设置最小字号；达到最小字号后使用横向 `scroll`，每次输入后自动滚动到末尾，同时保留闪烁光标。

8. 日期计算可能受到时区影响

如果直接用两个本地 `Date` 对象的毫秒值相减，夏令时切换可能得到非整数天。项目只提取年月日并使用 `Date.UTC` 生成日序号，再进行整数运算，从而保证日期间隔符合日历含义。

收获与体会

前四次实验一直围绕微信小程序进行，我已经比较熟悉微信开发者工具以及小程序的页面结构。这次第一次接触鸿蒙开发，刚打开DevEco Studio感觉很陌生，工程里的文件比以前多，运行方式也不一样。一开始模拟器还出现了连接问题，后来改用 X View 才顺利看到运行效果。随着修改次数增多，我也慢慢适应了新的开发环境。

和微信小程序相比，这次做出来的界面更像一款真正安装在手机上的应用。按键的大小、上下间距、文字位置和动画方向稍有不合适，在完整的手机界面中就会显得很明显。很多地方都是看到实际效果后才发现问题，再回去一点点调整。虽然这个过程花费的时间比预想中多，但每次修改后界面变得更顺眼，我都能直接感受到变化。

实验要求计算器要有个人特色，所以我从一个普通计算器入手，后面逐渐加入自己想到的功能和配色，过程有些煎熬和折腾，但完成后自我感觉比较良好！